

组件设计规范

本文档定义 WisePenView 项目组件的目录结构、命名约定和编写规范，作为后续开发可参考的规范。

一、设计理念

1. 职责划分

- 展示型组件 (Presentational)**：仅负责根据 Props 渲染 UI，无副作用、不直接调用 Service
- 容器型组件 (Container)**：负责数据获取、状态管理、调用 Service，并将数据/回调传给子组件
- 复合组件 (Composite)**：由多个子组件组合而成，子组件可拆分到独立目录

2. 三文件规范

每个独立组件遵循 `index.tsx + index.type.ts + style.module.less` 结构，保持结构清晰、职责单一。

3. 类型与样式

- 使用 TypeScript，**禁止** `any`，Props 和内部状态均有明确类型
- 样式使用 less + CSS Modules，**禁止内联样式**
- 优先使用 Ant Design 设计变量 (`var(--ant-*)`)

二、目录与文件规范

2.1 标准目录结构

代码块

```
1  src/components/
2  |   └─ Common/                # 通用基础组件
3  |   |   └─ QuotaBar/
4  |   |   |   └─ index.tsx      # 主逻辑
5  |   |   |   └─ index.type.ts  # Props 与类型定义
6  |   |   |   └─ style.module.less # 样式
7  |   └─ SelectedMemberList/
8  |      └─ index.tsx
```

```
9 | | | | index.type.ts
10 | | | | style.module.less
11 | | | | Group/ # 业务域组件
12 | | | | | | GroupCard/
13 | | | | | | | | index.tsx
14 | | | | | | | | index.type.ts
15 | | | | | | | | style.module.less
16 | | | | | | GroupModals/ # 一组相关 Modal, 可共享 style
17 | | | | | | | | index.ts # 统一导出
18 | | | | | | | | index.type.ts # 多个 Modal 的 Props
19 | | | | | | | | style.module.less
20 | | | | | | | | CreateGroupModal.tsx
21 | | | | | | | | JoinGroupModal.tsx
22 | | | | ...
```

2.2 文件职责

文件	职责
index.tsx	组件主逻辑，默认导出组件
index.type.ts	Props 接口、内部用到的类型（若非 *Props 可在同文件或单独 type 文件）
style.module.less	该组件的 CSS Modules 样式

2.3 类型文件命名

- 纯类型定义用 index.type.ts
- 若文件中包含 JSX 类型（如 React.ReactNode 等），可用 index.type.tsx

三、编写规范

3.1 Props 定义（index.type.ts）

```
代码块
1 import type { Group } from '@types/group';
2
3 export interface GroupCardProps {
4   group: Group;
5   onClick?: (group: Group) => void;
```

```
6 }
```

- 必填/选填通过 `?` 表达
- 从 `@/types` 引入业务类型，避免重复定义

3.2 组件实现 (`index.tsx`)

代码块

```
1 import React from 'react';
2 import { Card } from 'antd';
3 import type { GroupCardProps } from './index.type';
4 import styles from './style.module.less';
5
6 const GroupCard: React.FC<GroupCardProps> = ({ group, onClick }) => {
7   const handleClick = () => onClick?.(group);
8   return (
9     <div className={styles.card} onClick={handleClick}>
10       {group.name}
11     </div>
12   );
13 };
14
15 export default GroupCard;
```

- 使用 `React.FC<Props>` 或直接 `(props: Props) => JSX.Element`
- 从 `./index.type` 引入 Props, 从 `./style.module.less` 引入 styles
- 组件使用 `export default` 导出

3.3 样式编写 (`style.module.less`)

代码块

```
1 .card {
2   border: 1px solid var(--ant-color-border);
3   border-radius: var(--ant-border-radius-lg);
4   padding: var(--ant-padding-md);
5 }
6
7 .metaDescription {
8   display: flex;
9   align-items: center;
10  gap: var(--ant-margin-xxs);
```

```
11 }
```

- 类名使用 camelCase
- 优先使用 Ant Design 变量：`var(--ant-color-*)`、`var(--ant-margin-*)`、`var(--ant-padding-*)`、`var(--ant-font-size-*)` 等
- 需要覆盖 Ant 组件样式时，使用 `:global(.ant-xxx)` 限定作用域

3.4 复合组件与子组件

- 子组件放在独立目录，如 `MemberList/MemberListTable/`、`MemberList/MemberListToolbar/`
- 父组件通过 `index.ts` 统一导出，如 `GroupModals/index.ts` 导出多个 Modal
- 共享样式可放在父级 `style.module.less`，子组件按需使用

四、Modal 组件规范

- Props 统一包含：`open`、`onCancel`、`onSuccess?`
- 关闭时调用 `onCancel()`，操作成功时先 `onSuccess?.()` 再 `onCancel()`
- 表单用 `Form.useForm`，关闭/取消时 `form.resetFields()`
- 错误处理：catch 后 `message.error(parseErrorMessage(err, fallback))`，不直接 `console.error` 展示给用户

五、命名约定

类型	规范	示例
组件目录	PascalCase	<code>QuotaBar</code> 、 <code>GroupCard</code>
组件文件	<code>index.tsx</code>	主入口
Props 接口	<code>{ComponentName}Props</code>	<code>QuotaBarProps</code>
样式类名	camelCase	<code>quotaBar</code> 、 <code>metaDescription</code>
常量/枚举	UPPER_SNAKE 或 PascalCase	<code>GROUP_TYPE</code> 、 <code>ROLE_LABEL</code>

六、性能与可维护性

- **useMemo / useCallback**: 仅在依赖明确、确有性能收益时使用（如大列表、复杂计算）
- **useEffect**: 需处理清理函数（订阅、定时器等），防止内存泄漏
- **子组件拆分**: 逻辑复杂或可复用时拆成独立组件，避免单文件过长

七、规范检查清单

- ☐ 组件目录包含 index.tsx、index.type.ts、style.module.less
- ☐ Props 在 index.type.ts 中定义，无 any
- ☐ 样式使用 less + CSS Modules，无非必要的内联样式
- ☐ 优先使用 Ant Design 设计变量
- ☐ Modal 具备 open、onCancel、onSuccess? 约定
- ☐ 涉及 Service 调用时，错误由 View/Modal 层 catch 并 parseErrorMessage 展示